Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

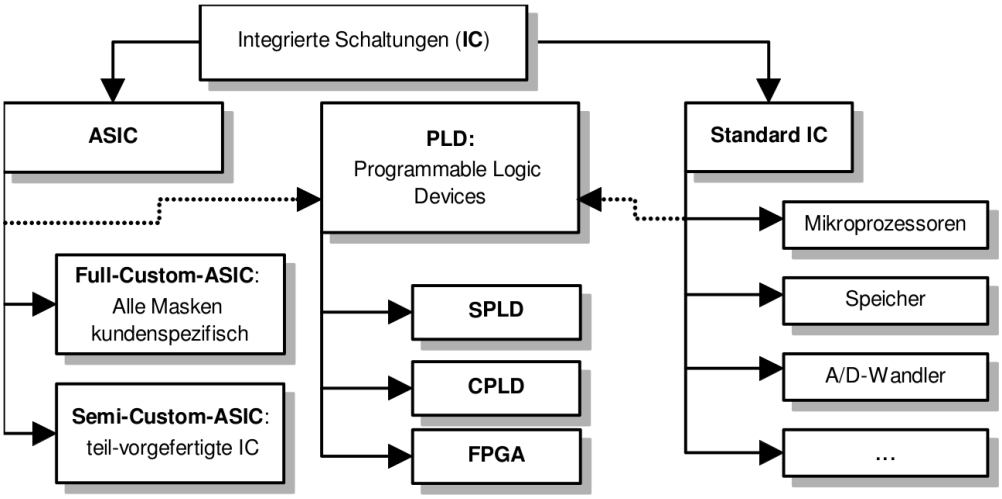
Kapitel 05 - Synthese und Implementierung synchroner Schaltungen

Prof. Dr.-Ing. Guillermo Payá Vayá, 9. Januar 2024

Auswahl von Implementierungsformen für integrierte Schaltungen

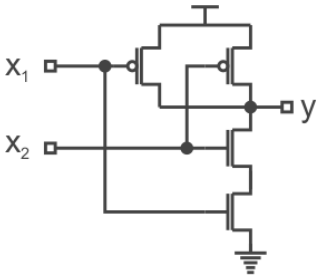
Digitale integrierte Schaltungen

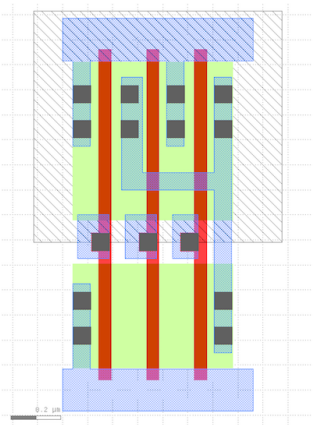
- VHDL-Schaltungsbeschreibungen werden in digitalen integierten Schaltungen (engl.: Integrated Circuit (IC)) realisiert.
- Eine Entwicklung von Schaltungen mit VHDL ohne ein genaueres Verständnis der Hardware ist nur schwer möglich.



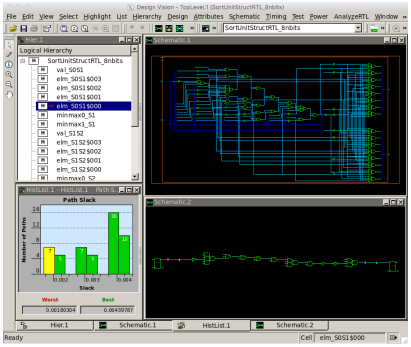
Application-Specific IC (ASIC)

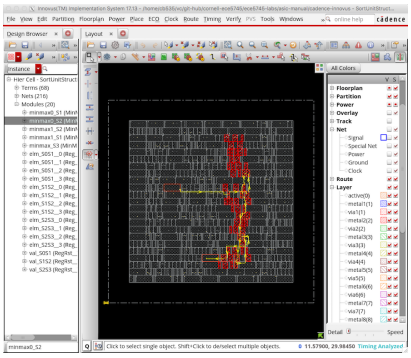
Full-Custom ASIC





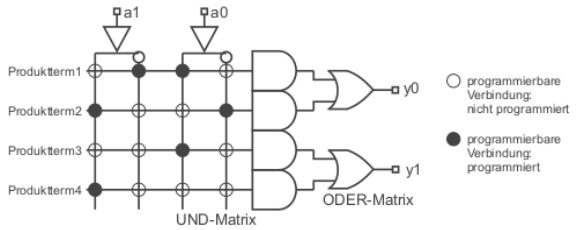
Semi-Custom ASIC



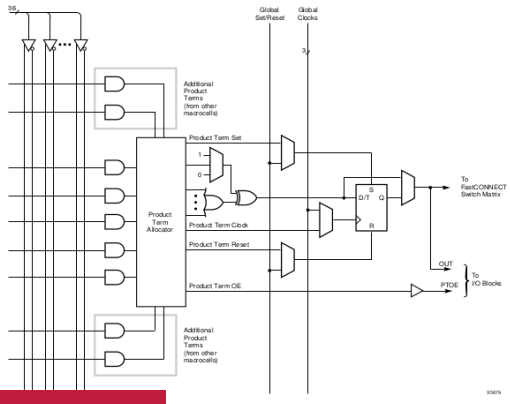


Programmierbare Logikbausteine (PLD)

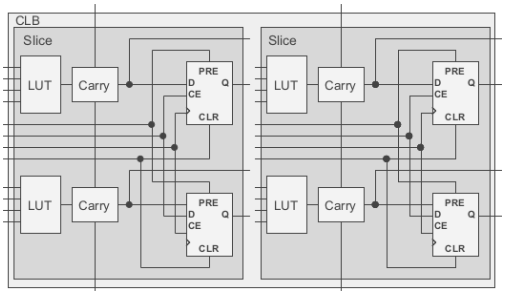
Simple PLD

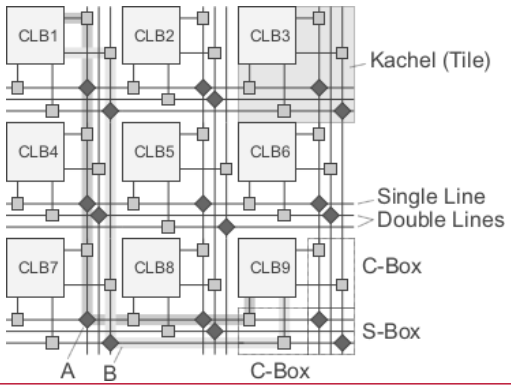


Complex PLD



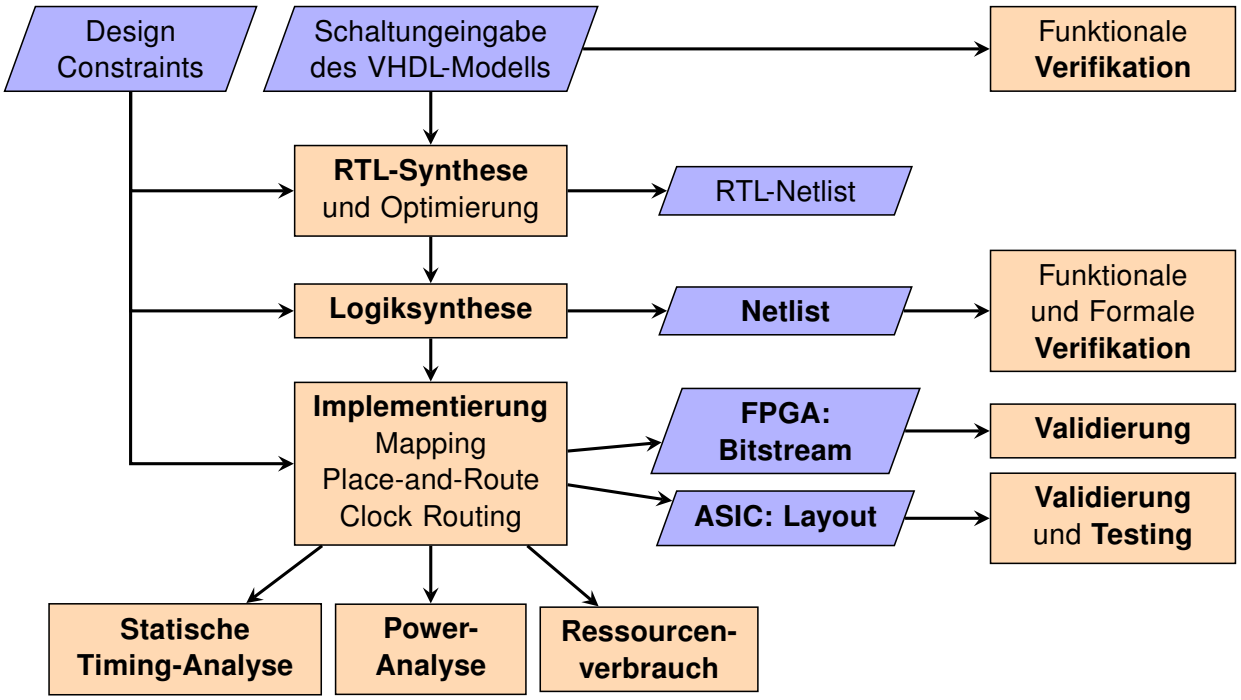
Field Programmable Gate Arrays (FPGA)





Generische Hardware-Entwurfsmethodik

Design Flow (I)



Design Flow (II)

- Die **Eingabe des digitalen Systems** erfolgt mit einem Texteditor als HDL-Modell. Weiter unterstützte Modellierungsverfahren sind ein *Blockshaltplaneditor* für einen IP-zentrierten Entwurf sowie die Eingabe von *C- oder C++-Code*.

  → **Kap 8 - High-Level-Synthesis**

- In einer oder mehreren **funktionalen Simulationen** wird überprüft, ob das Modell die funktionalen Spezifikationen des Systems erfüllt (**Funktionale Verifikation**). Dies erfolgt auf Basis verschiedener Simulationstests, die als Testvektoren bezeichnet werden. In den Testvektoren erhalten alle Eingangssignale einen bestimmten Wert (Stimulus) und die Testergebnisse (Response) können mit den zu erwartenden Werten verglichen werden. Die Testvektoren und der Vergleich können in einer sogenannten Testbench zusammengefasst werden, die ebenfalls als VHDL-Code formuliert werden kann.

  → **Kap 3.7 - Simulation und Testbench**

Design Flow (III)

- Während des **RTL-Synthese**, auch als *Elaboration* bezeichneten Analyseschritts wird das komplette HDL-Modell, welches aus mehreren HDL-Dateien bestehen kann, in eine zunächst noch weitgehend *technologieunabhängige Netzliste* (**RTL-Netlist**) von Komponenten der Digitaltechnik (kombinatorische Logikkomponenten, Flipflops etc.) überführt. Komplexere, häufig vorkommende Komponenten wie z.B. Addierer werden in geeignete Schaltplansymbole transformiert.

  → **Kap 5 - Synthese und Implementierung synchroner Schaltungen**

- Nun erfolgt die eigentliche **Logiksynthese**, also die Überführung des **RTL-Synthese** ergebnisses in die auf der vorgesehenen ASIC- oder FPGA-Familie existierenden Hardwareressourcen wie z.B. Look-up-Tabellen, Flipflops etc. für FPGA und Standard-Cells für ASIC.

  → **Kap 5 - Synthese und Implementierung synchroner Schaltungen**

  → **Kap 6 - FPGA-Entwurftechnik**

  → **Kap 7 - ASIC-Design**

Design Flow (IV)

- Die Verifikation durch **funktionale Verifikation** oder **formale Methoden** des, während der *Logiksynthese* erzeugten, Schaltplans ist im Entwicklungsablauf ein außerordentlich wichtiger Schritt, weil es bei einer nicht synthese-gerechten oder semantisch fehlerhaften Modellierung des HDL-Codes zu einem Fehlverhalten der Schaltung kommen wird.

- Spätestens während der **Implementierung** müssen die **Constraints** angeben werden. Dazu gehören insbesondere die genaue Bausteinbezeichnung, die Pinbelegung mit den Schnittstellensignalen sowie die Taktrate(n), mit denen das System betrieben werden soll. Auf der Grundlage der, während der *Synthese* erstellten, Netzliste und der Constraints werden sukzessive die verschiedenen Schritte abgearbeitet, die bei *FPGAs* zusammen genommen als *Physical Design* bezeichnet werden. Alternativ wird dafür auch der aus der *ASIC-Welt* stammende Begriff *Backend Design* verwendet.

  → **Kap 6 - FPGA-Entwurftechnik** und **Kap 7 - ASIC-Design**

Design Flow (V)

- Eine **statische Timing-Analyse** bedeutet die systematische Suche nach dem kritischen Pfad des Systems, also der längsten Laufzeit kombinatorischer Logik zwischen zwei getakteten Speicherelementen. Dieser Pfad legt bekanntlich die maximale Taktfrequenz fest, mit der das System betrieben werden kann.
  → **Kap 5 - Synthese und Implementierung synchroner Schaltungen**
  → **Kap 6 - FPGA-Entwurftechnik**
  → **Kap 7 - ASIC-Design**
- Mit einer Post-Implementation **Validierung** und **Testing** kann die Funktionalität des resultierendes FPGA-Design oder ASIC-Design verifiziert werden.

Synthese

Synthese

Synthese (Kesel2018)

Automatische Umsetzung einer Darstellung der Schaltung auf einer Abstraktionsebene in eine Darstellung auf der nächst tieferen Abstraktionsebene

- **High-Level Synthese:** Umsetzung von der algorithmischen Ebene auf eine *technologieunabhängigde* Register-Transfer-Ebene (auch bezeichnet als *Architektursynthese* oder *Behavioral Synthesis*).
- **RTL-Synthese:** Umsetzung von der Register-Transfer-Ebene oder HDL-Code auf eine **noch** *technologieunabhängigde* Darstellung (**RTL-Netlist**) mit Logikgattern und Makros (auch bezeichnet als *RTL-Analysis* oder *RTL-Translation*).
- **Logiksynthese** Umsetzung von der technologieunabhängigen RTL-Netlist auf die technologieabhängige Gatterebene (**Netlist**)

RTL-Synthese (I): Inference

- Das Ergebnis der RTL-Synthese ist entscheidend für die Qualität der Gatterrealisierung: *Eine ineffiziente VHDL-Beschreibung kann erfahrungsgemäß auch durch die nachfolgende Logikoptimierung nicht mehr in eine leistungsfähige Hardware umgesetzt werden.*

```
entity comparator is
  port ( A, B     : in  std_ulogic_vector(1 downto 0);
         clk, rst : in  std_ulogic;
         Y        : out std_ulogic);
end entity comparator;
architecture RTL of compare is
  signal q : std_ulogic;
begin
  comp: process (clk, rst)
  begin
    if rst = '1' then q <= '0';
    elsif clk'event and clk = '1' then
      if A > B then q <= '0';
      else          q <= '1';
      end if;
    end if;
  end process comp;
  Y <= q;
end architecture RTL;
```

RTL-Synthese (II): Operatoren und Datetyp

- Der *relationale Größer-Operator* (>) wird zunächst durch ein entsprechendes *Black Box-Makro* dargestellt, das erst während der Logiksynthese und Technologieabbildung durch eine für die Zieltechnologie optimierte Implementierung ersetzt wird.
- Der Vorgang des Erkennens von Makros während der RTL-Synthese des HDL-Codes wird im Englischen als **inference** bezeichnet (dt.: Schlussfolgerung) und daher häufig deutschsprachig als *Inferenz*.

→ Neben den relationalen und arithmetischen Operatoren sowie den Schiebeoperatoren können beispielsweise auch Flip-flops, Latches, Register, Tristates/Busse, Multiplexer, Decoder, Zähler, Schaltwerke, RAM und ROM aus dem HDL-Code *inferiert* werden.

RTL-Synthese (III): RTL-Synthese-Report

```
Synthesizing Unit <compare>.
    Related source file is compare.vhd.
    Found 2-bit comparator greater for signal <$n0002> created at line 23.
    Found 1-bit register for signal <q>.
    Summary:
        inferred   1 D-type flip-flop(s).
        inferred   1 Comparator(s).
Unit <compare> synthesized.

Macro Statistics
# Registers                                  : 1
  1-bit register                             : 1
# Comparators                                : 1
  2-bit comparator greater                   : 1
```

- Der Vorteil der Inferenz eines Makros ist es, dass der HDL-Code technologieunabhängig und damit portabel ist und dennoch eine optimierte Implementierung gefunden werden kann.
  → Effizienter HDL-Code zeichnet sich unter anderem dadurch aus, dass ein großer Teil des Codes zur Inferenz von Makros führt.

RTL-Synthese (IV): Instanzierung von Makros

- Kann eine bestimmte Makro-Funktion nicht über Inferenz gefunden werden, so ist es möglich, eine optimierte, technologieabhängige Implementierung im HDL-Code zu instanzieren. Hierzu bieten die FPGA- und ASIC-Hersteller so genannte **Generatoren**.
  - Ein Beispiel für ein solches Makro wäre die ganzzahlige Division oder Speicherkomponenenten (z.B. RAM, FIFO, ...)
- Nachteilig an der Instanzierung von Makros im HDL-Code ist es, dass der Code nun technologieabhängig und damit nicht mehr einfach portierbar ist.

Randbedingungen (Constraints)

- Bei den meisten Synthesetools kann der Benutzer den Synthese- oder Implementierungsprozess durch **Constraints** beeinflussen.
  - Betriebsbedingungen (engl.: operating conditions)
  - Explizite Geschwindigkeitsanforderungen (z.B. Taktfequenz)
  - Hohe Geschwindigkeit oder kleine (Chip-)fläche
  - FSM-Analysis: One-Hot- oder Binäre Codierung
  - Verwendung von Arithmetik-Hardwarekomponenten im Datenpfad
  - Ressource-Sharing
  - Manipulation der Designhierarchie

```
architecture RTL of COMPONENT is
begin
  P1: process(A, B, C, D, SEL)
  begin
    if SEL='1' then S1 <= A + B;
    else            S1 <= C + D;
    end if;
  end process P1;
end architecture RTL;
```

Logiksynthese (I): Makros

- Nach der Übersetzung des VHDL-Codes und der Angabe der Randbedingungen muss das Design in eine Gatternetzliste in der gewählten Zielbibliothek umgesetzt werden.
- Bei der **Logiksynthese** wird zunächst die technologieunabhängige Darstellung der Schaltung gemäß der Syntheseziele optimiert.

1) Für ein Makro können verschiedene Implementierungen (schnelle Implementierung, Implementierung mit geringster Anzahl an Ressourcen) existieren. Welche davon ausgewählt wird, hängt im Wesentlichen von den vom Benutzer vorgegebenen Randbedingungen (**Constraints**) für die Logiksynthese und der gewählten Zieltechnologie ab.

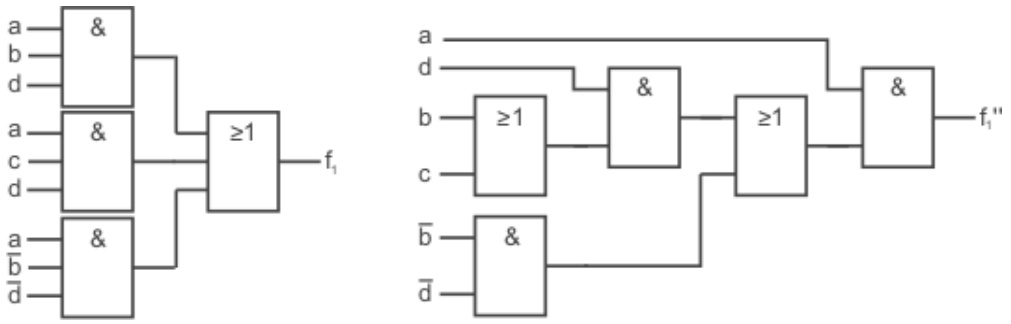
Logiksynthese (II): Logikoptimierung

2) Logikoptimierung und Technologieabbildung hängen zusammen und werden in den Synthesewerkzeugen zumeist auch in einem Vorgang ausgeführt; dieser wird häufig als Optimierung bezeichnet (engl.: optimization).

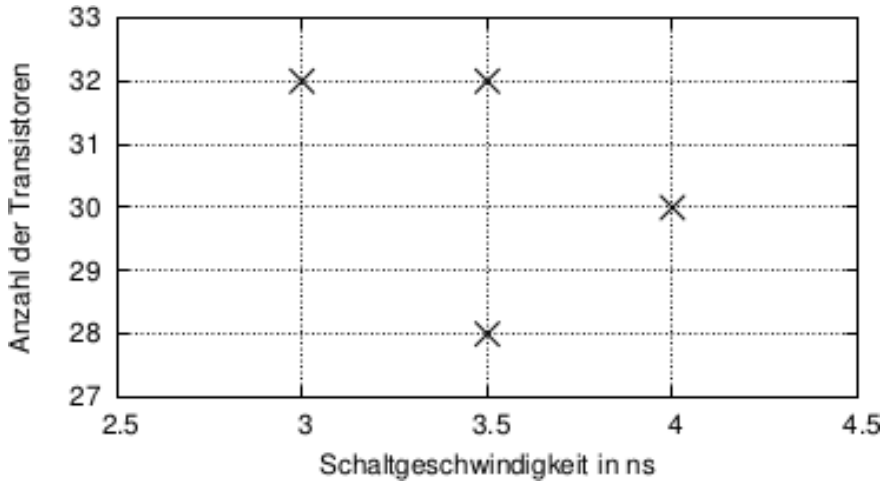
- Bei den Verfahren zur Logikoptimierung muss zunächst unterschieden werden in Verfahren zur:
  - Zweistufigen Optimierung (z.B. *Quine-McCluskey-Verfahren* oder *KV-Diagramm*), die als Ergebnis eine zweistufige disjunktive Minimalform haben,
  - und mehrstufigen Optimierung (engl.: *multilevel logic synthesis*).
- Logikoptimierung ein schweres algorithmisches Problem, welches ein ungünstiges Rechenzeitverhalten aufweist. Daher werden bei der Logikoptimierung ebenfalls heuristische Verfahren verwendet, die zwar nicht garantiert das Optimum finden, aber doch akzeptable Lösungen in kürzerer Rechenzeit.

Logiksynthese (III): Beispiel

Schaltung: $Y = (a \wedge d \wedge (c \vee b)) \vee (a \wedge \overline{(b \vee d)})$



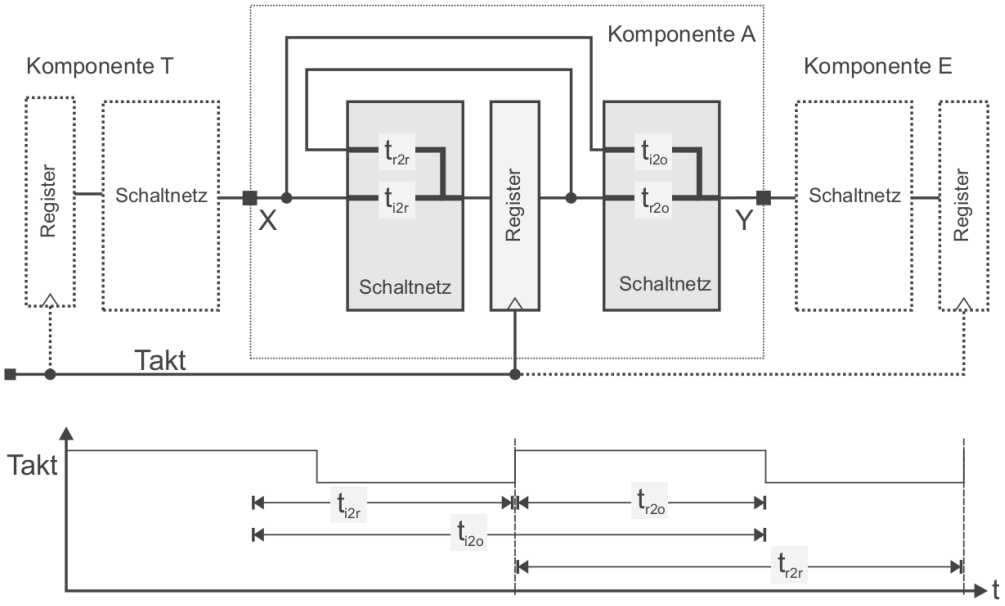
Logiksynthese (IV): Entwurfsraum (Design Space)



- Zusammenhang zwischen dem Hardwareaufwand (Ressourcenbedarf) – als Anzahl der Transistoren – und der Verzögerungszeit – auf dem kritischen Pfad – dar.

Statische Timing-Analyse

Zeitliche Randbedingungen (I)



Zeitliche Randbedingungen (II)

- Als zeitliche Randbedingungen sind im Wesentlichen vier Zeiten anzugeben:
  1) **Register-Register** t_{r2r}: Diese Zeit bestimmt die maximale Taktfrequenz $f_{max} = \frac{1}{t_{r2r}}$, mit der die Schaltung später betrieben wird. Die Verzögerungszeit auf dem längsten Pfad vom Register durch das Schaltnetz zum Register darf diesen Wert nicht überschreiten, wobei zu beachten ist, dass die Verzögerungszeit und die *Setup-Zeit* der Flipflops noch davon subtrahiert werden müssen.
  2) **Input-Register** t_{i2r}: Das ist die Zeit, die für die Komponente vom Eingang bis zum Register zur Verfügung steht. Zusammen mit der Zeit $t_{r2o,T}$, die die treibende Komponente T vom Register bis zum Ausgang benötigt, darf die Zeit nicht größer sein als die Taktperiode $t_{r2o,T} + t_{i2r} \leqslant t_{r2r}$.

Zeitliche Randbedingungen (III)

  3) **Register-Output** t_{r2o}: Das ist die Zeit, die für die Komponente vom Register bis zum Ausgang zur Verfügung steht. Zusammen mit der Zeit $t_{r2o,E}$, die die treibende Komponente E vom ihrem Eingang bis zum Register benötigt, darf die Zeit nicht größer sein als die Taktperiode $t_{i2r} + t_{r2o,E} \leqslant t_{r2r}$.
  4) **Input-Output** t_{i2o}: Diese Zeit steht für Pfade zur Verfügung, die nicht über ein Register führen. Bei der Einbettung ins Gesamtsystem muss gelten: $t_{r2o,T} + t_{i2o} + t_{r2o,E} \leqslant t_{r2r}$.

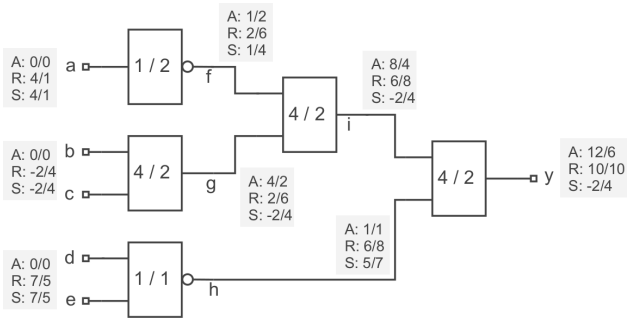
- Eine weitere wichtige Angabe ist der so genannte *Taktversatz* (engl.: **clock skew**): Bedingt durch die Realisierung des Taktnetzes auf dem Chip können die Taktflanken zu unterschiedlichen Zeitpunkten an den Flipflops ankommen. Wird der Taktversatz bei den Randbedingungen als *Constraint* angegeben, so kann er während der Synthese berücksichtigt werden.

Statische Timing-Analyse

- Die statische Timing-Analyse ist insbesondere nach der Platzierung und Verdrahtung des Designs auf dem Chip notwendig.
- Der Begriff *statisch* bedeutet hier, dass die Analyse von den Belegungen der Eingangssignale und auch von der Funktion der Gatter und der Schaltung unabhängig ist.
  - Grundlage für die Analyse ist nur die Topologie der Schaltung, also die Anordnung und Verdrahtung der Gatter.
  - Durchführung der statische Timing-Analyse nach dem häufig verwendeten CPM-Verfahren (*Critical Path Method*)
- Eine andere Möglichkeit der Timing-Analyse ist eine Simulation des Gattermodells der Schaltung: *dynamische Timing-Analyse*

Critical Path Method (I)

- In der ersten Phase der Timing-Analyse werden zunächst die Verzögerungszeiten entlang aller Pfade durch die Schaltung bestimmt.



- *A:(r,f)* - *Ankunftszeit* (engl.: arrival time) an einem Eingang, internen Netz oder einem Ausgang der Schaltung. Von *r* ist für die steigender Flanke und *f* für die fallender Flanke.
- t_r und t_f definieren die Verzögerungszeit eines Gatters.
- Nicht invertierendes Gatter: $A_{o,r} = max_{i=1..n}\{A_{i,r}\} + t_r$ und $A_{o,f} = max_{i=1..n}\{A_{i,f}\} + t_f$
- Invertierendes Gatter: $A_{o,r} = max_{i=1..n}\{A_{i,f}\} + t_r$ und $A_{o,f} = max_{i=1..n}\{A_{i,n}\} + t_f$

Critical Path Method (II)

- *R:(r,f)* - Ankunftszeit mit den vom Benutzer durch die Randbedingungen vorgegebenen Zeiten R (engl.: *required time*) am Ausgang verglichen werden.
- *S:(r,f)* - Zeitunterschied (engl.: *Slack*) zwischen der vorgegebenen Ankunftszeit und der tatsächlichen Ankunftszeit
  $S_r = R_r - A_r$ und $S_f = R_f + A_f$
- Bei einem *positiven Slack* ist die Schaltung schneller als spezifiziert, bei einem *negativen Slack* ist die Schaltung zu langsam und bei einem Slack von null entspricht die Verzögerungszeit gerade den Vorgaben.
- Der so genannte **kritische Pfad** kann durch ein Zurückverfolgen (engl.: trace back, backtrack) vom Ausgang zu den Eingängen ermittelt werden.

Das Problem des *falschen Pfades*

- Bei der statischen Timing-Analyse werden einige Vereinfachungen gemacht:
  - Die Boole'sche Funktion der Gatter und die binären Werte an den Eingängen der Schaltungen werden nicht berücksichtigt.
  - Damit wird das Verfahren sehr einfach und kann auch für große Schaltungen mit akteptabler Rechenzeit durchgeführt werden.
- Es ist allerdings möglich, dass über den so gefundenen kritischen Pfad – aufgrund der logischen Funktion der Schaltung – nie ein Signalwechsel propagiert wird. Dies wird als **falscher Pfad** bezeichnet.
  - In diesem Fall ist die Aussage der statischen Timing-Analyse pessimistisch: Die Schaltung wird tatsächlich höher getaktet werden können.

Literatur

- Frank Kesel, "FPGA Hardware-Entwurf. Schaltungs- und System-Design mit VHDL und C/C++", Walter de Gruyter GmbH, 2018
- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.